O'ZBEKISTON RESPUBLIKASI

OPERATSION TIZIMLAR fanidan HISOBOT

1-topshiriq: C tili asoslari, kompilyatsiya va pointerlar

2-topshiriq: Pointerlar va dinamik xotira (malloc/free)

Fan: Operatsion tizimlar

Topshiriq raqami: 1 va 2

Variant: 18

F.I.Sh.: Xolboyev Anvarbek

Guruh: SRM003-2

O'qituvchi: Bekimmetova Durdona

2026

1-TOPSHIRIQ. C tili asoslari, kompilyatsiya va pointerlar

Variant 18. Mavzu: siklli kalkulyator. Muhit: Linux (WSL2), GCC. Kompilyatsiya: `gcc -Wall -Wextra -o main main.c -lm` (ogohlantirishsiz). `-lm` qo'shilgan, chunki `fmod()` <math.h> da.

A qism. "Hello World" ni 4 bosqichda kompilyatsiya qilish

Manba fayl `hello.c` alohida bosqichlarda quyidagi buyruqlar bilan kompilyatsiya qilindi:

```
gcc -E hello.c -o hello.i   # 1) preprocessing
gcc -S hello.i -o hello.s   # 2) kompilyatsiya (assembler kodiga)
gcc -c hello.s -o hello.o   # 3) assemblerlash (obyekt fayl)
gcc hello.o -o hello        # 4) linking (bajariluvchi fayl)
./hello
```

Fayl	Hajmi	Bosqich	Nima ekani
`hello.c`	83 bayt	manba	Dasturchi yozgan matn: #include, main va printf chaqiruvi.
`hello.i`	21 315 bayt (819 qator)	`gcc -E`	Preprocessor natijasi. #include <stdio.h> o'rniga stdio.h ning butun matni qo'yilgan, izohlar olib tashlangan. Hali ham C kodi, shuning uchun hello.c dan ancha katta.
`hello.s`	668 bayt	`gcc -S`	Assembler matni (x86-64). Matn ko'rinishida; printf("...\n") kompilyator tomonidan puts chaqiruviga almashtirilgan.
`hello.o`	1 504 bayt	`gcc -c`	Obyekt fayl (ELF, relocatable): mashina kodi, lekin puts ning manzili hali ma'lum emas, shuning uchun bajarib bo'lmaydi.
`hello`	15 960 bayt	linking	Bajariluvchi fayl (ELF). Linker puts ni libc ga bog'lagan va ishga tushirish kodi (_start) qo'shilgan; shuning uchun .o dan katta.

```
● ● ● A qism: natija
$ ./hello
Hello, World!
```

.o va bajariluvchi fayl farqi: .o fayl bitta manba faylning mashina kodi bo'lib, tashqi funksiyalar (puts) manzillari to'ldirilmagan. Linking bosqichida linker kutubxonalarni bog'laydi, manzillarni to'ldiradi va ishga tushirish kodini qo'shadi, natijada OS yuklab ishga tushira oladigan fayl hosil bo'ladi.

B qism. Siklli kalkulyator (variant 18)

1-masala (C asoslari). Dastur `q` kiritilguncha ikki son va amal (+ - * / %) so'raydi va natijani chiqaradi. `/` va `%` amallarida nolga bo'lish tekshiriladi. Sonlar `fgets` + `strtod` bilan o'qiladi; harf, bo'sh qator, nan/inf rad etiladi. Amal faqat + - * / % dan biri bo'lishi shart.

2-masala (pointerlar). Amal belgisiga qarab `double (*amal)(double, double)` funksiya pointeri tanlanadi va hisob shu pointer orqali chaqiriladi. `%` amali C da `double` bilan ishlamagani uchun `fmod()` orqali yozilgan. Kamida bitta o'zgaruvchining manzili `%p` bilan chiqariladi.

Asosiy kod qismlari va izohlari

1) Amal funksiyalari. Hammasi bir xil imzoga ega (ikkita double olib double qaytaradi), shuning uchun bitta pointer turi bilan ishlash mumkin:

```
static double qosh(double a, double b)     { return a + b; }
static double ayir(double a, double b)     { return a - b; }
static double kopaytir(double a, double b) { return a * b; }
static double bol(double a, double b)      { return a / b; }
static double qoldiq(double a, double b)   { return fmod(a, b); }
```

2) Funksiya pointerini tanlash. `amal_tanla` amal belgisiga mos funksiya manzilini qaytaradi; noto'g'ri belgi bo'lsa `NULL` (shu bilan amal tekshiriladi):

```
static double (*amal_tanla(char belgi))(double, double)
{
    switch (belgi) {
    case '+': return qosh;
    case '-': return ayir;
    case '*': return kopaytir;
    case '/': return bol;
    case '%': return qoldiq;
    default:  return NULL;
    }
}
```

3) Nolga bo'lish tekshiruvi va pointer orqali chaqirish. Tekshiruv funksiya chaqirilishidan oldin bajariladi:

```
double (*amal)(double, double);          /* funksiya pointeri */
...
if ((belgi == '/' || belgi == '%') && b == 0.0) {
    printf("Xato: nolga bo'lish mumkin emas.\n");
    continue;
}
amal = amal_tanla(belgi);   /* pointer amal belgisiga qarab tanlanadi */
natija = amal(a, b);        /* funksiya pointer orqali chaqiriladi */
```

4) Kiritishni tekshirish. `son_oqish()` qatorni `fgets` bilan o'qib, `strtod` orqali tahlil qiladi: son boshlanmasa, oxirida ortiqcha belgi qolsa (masalan `abc`, `5x`) yoki natija `nan/inf` bo'lsa, xabar chiqarib qayta so'raydi. Bu `scanf` dan ishonchliroq: xato kiritishda dastur qotib qolmaydi.

Test natijalari

Quyidagi testlar dasturning haqiqiy ishga tushirilishidan olingan (kiritilgan qiymatlar terminalda ko'rinadi).

1-test: oddiy amallar (+, *, -)

```
● ● ● ./main
=== SIKLLI KALKULYATOR ===
(to'xtatish uchun son o'rniga q kiriting)

Birinchi son: 10
Ikkinchi son: 3
Amal (+ - * / %): +
Natija: 10 + 3 = 13
--- Xotira manzillari ---
natija o'zgaruvchisi &natija = 0x7ffe530ea168
pointer o'zgaruvchisi &amal   = 0x7ffe530ea170

Birinchi son: 7.5
Ikkinchi son: 2
Amal (+ - * / %): *
Natija: 7.5 * 2 = 15
--- Xotira manzillari ---
natija o'zgaruvchisi &natija = 0x7ffe530ea168
pointer o'zgaruvchisi &amal   = 0x7ffe530ea170

Birinchi son: 20
Ikkinchi son: 8
Amal (+ - * / %): -
Natija: 20 - 8 = 12
--- Xotira manzillari ---
natija o'zgaruvchisi &natija = 0x7ffe530ea168
pointer o'zgaruvchisi &amal   = 0x7ffe530ea170

Birinchi son: q
Dastur tugadi.
```

2-test: nolga bo'lish (/ va %), qoldiq va bo'lish

```
● ● ● ./main
=== SIKLLI KALKULYATOR ===
(to'xtatish uchun son o'rniga q kiriting)

Birinchi son: 10
Ikkinchi son: 0
Amal (+ - * / %): /
Xato: nolga bo'lish mumkin emas.

Birinchi son: 10
Ikkinchi son: 0
Amal (+ - * / %): %
Xato: nolga bo'lish mumkin emas.

Birinchi son: 10
Ikkinchi son: 4
Amal (+ - * / %): %
Natija: 10 % 4 = 2
--- Xotira manzillari ---
natija o'zgaruvchisi &natija = 0x7ffc9fb98738
pointer o'zgaruvchisi &amal   = 0x7ffc9fb98740

Birinchi son: -7
Ikkinchi son: 3
Amal (+ - * / %): %
Natija: -7 % 3 = -1
--- Xotira manzillari ---
natija o'zgaruvchisi &natija = 0x7ffc9fb98738
pointer o'zgaruvchisi &amal   = 0x7ffc9fb98740
```

```
Birinchi son: 12
Ikkinchi son: 5
Amal (+ - * / %): /
Natija: 12 / 5 = 2.4
--- Xotira manzillari ---
natija o'zgaruvchisi &natija = 0x7ffc9fb98738
pointer o'zgaruvchisi &amal   = 0x7ffc9fb98740

Birinchi son: q
Dastur tugadi.
```

3-test: XATO kiritishlar (harf, bo'sh qator, noto'g'ri amal belgisi)

```
● ● ● ./main
=== SIKLLI KALKULYATOR ===
(to'xtatish uchun son o'rniga q kiriting)

Birinchi son: abc
Xato: son kiriting (masalan 12, -3.5) yoki q.

Birinchi son:
Xato: son kiriting (masalan 12, -3.5) yoki q.

Birinchi son: 5
Ikkinchi son: 2.5
Amal (+ - * / %): 3
Xato: amal faqat + - * / % dan biri bo'lishi kerak.
Amal (+ - * / %): ^
Xato: amal faqat + - * / % dan biri bo'lishi kerak.
Amal (+ - * / %): ab
Xato: amal faqat + - * / % dan biri bo'lishi kerak.
Amal (+ - * / %): -
Natija: 5 - 2.5 = 2.5
--- Xotira manzillari ---
natija o'zgaruvchisi &natija = 0x7ffefbee5378
pointer o'zgaruvchisi &amal   = 0x7ffefbee5380

Birinchi son: q
Dastur tugadi.
```

Test	Kiritish	Kutilgan natija	Natija
1	10 + 3; 7.5 * 2; 20 - 8	13; 15; 12	To'g'ri
2	10 / 0; 10 % 0; 10 % 4; -7 % 3; 12 / 5	Nolga bo'lish xabari (ikki marta); 2; -1; 2.4	To'g'ri
3	abc, bo'sh qator; amal: 3, ^, ab	Barcha xato kiritishlar xabar bilan rad etilib, qayta so'raladi	To'g'ri

%p bo'yicha izoh. Dastur ikki manzil chiqaradi: `&natija;` (hisob natijasi saqlanadigan double o'zgaruvchi) va `&amal;` (funksiya pointeri o'zgaruvchisining o'z manzili). Ikkalasi ham 8 baytli `double` va pointer bo'lgani uchun manzillar farqi 8 bayt (masalan ...a168 va ...a170). Manzillar 0x7ff... bilan boshlanadi, ya'ni ular stek sohasida. Manzillar har ishga tushirishda boshqacha bo'ladi (ASLR), chunki operatsion tizim stekni tasodifiy joyga joylashtiradi.

Xulosa (1-topshiriq)

Ishda C kompilyatsiyasining to'rt bosqichi (preprocessing, kompilyatsiya, assemblerlash, linking) amalda bajarildi va har bir bosqich fayli o'rganildi. Siklli kalkulyator yozildi: amal funksiya pointeri orqali tanlanib chaqiriladi, `%` amali `fmod()` bilan bajariladi, nolga bo'lish va noto'g'ri kiritishlar xabar bilan rad etiladi. %p orqali o'zgaruvchilar manzillari ko'rsatildi.

GitHub repozitoriy: https://github.com/<username>/OS_T1_V18_Xolboyev

2-TOPSHIRIQ. Pointerlar va dinamik xotira (malloc/free)

Variant 18. Mavzu: 100+ xonali ikkita katta sonni satr sifatida qabul qilib, yig'indisini yangi dinamik satrda qaytarish.

Majburiy talablar: ma'lumot hajmi dastur ishlayotganda aniqlanadi (satr uzunligi oldindan noma'lum); xotira faqat dinamik ajratiladi; har bir malloc/realloc natijasi NULL ga tekshiriladi; dastur oxirida barcha xotira bo'shatiladi; kamida bitta funksiya argumentni pointer orqali o'zgartiradi.

Asosiy kod qismlari va izohlari

1) Satrni dinamik o'qish va argumentni pointer orqali o'zgartirish. Satr uzunligi noma'lum, shuning uchun bufer kichik hajmdan boshlanib, to'lganda `realloc` bilan 2 barobar kengayadi. Funksiya `char **satr` oladi va yangi ajratilgan satr manzilini chaqiruvchining o'zgaruvchisiga yozadi (argument pointer orqali o'zgaradi):

```
static int qator_oqish(char **satr)
{
    size_t sigim = BOSH_HAJM, n = 0;
    char *buf = malloc(sigim);
    int c;
    if (buf == NULL)
        return 0;                          /* xotira yetmadi */
    while ((c = getchar()) != '\n' && c != EOF) {
        if (n + 1 >= sigim) {
            char *yangi = realloc(buf, sigim * 2);
            if (yangi == NULL) { free(buf); return 0; }
            buf = yangi;
            sigim *= 2;
        }
        buf[n++] = (char)c;
    }
    ...
    buf[n] = '\0';
    *satr = buf;                           /* chaqiruvchining pointerini o'zgartiramiz */
    return 1;
}
```

2) Katta sonlarni qo'shish. Sonlar `int` ga sig'maydi, shuning uchun maktabdagi ustun usulida, o'ngdan chapga raqamma-raqam qo'shiladi va ko'chirish (carry) saqlanadi. Natija teskari tartibda `tmp` ga yig'iladi, so'ng yangi `natija` satriga to'g'rilab ko'chiriladi; `tmp` darhol `free` qilinadi. Elementlarga indeks emas, pointer arifmetikasi bilan murojaat qilinadi:

```
for (i = 0; i < uzun; i++) {
    int s = carry;
    if (i < la) s += *(a + la - 1 - i) - '0';
    if (i < lb) s += *(b + lb - 1 - i) - '0';
    *(tmp + i) = (char)('0' + s % 10);
    carry = s / 10;
}
```

3) Kiritishni tekshirish. `raqamlarmi()` satr bo'sh emasligini va faqat '0'..'9' dan iboratligini tekshiradi; harf, minus ishorasi, bo'shliq va bo'sh qator rad etiladi. Boshidagi ortiqcha nollar `nolsiz()` bilan o'tkazib yuboriladi ("0012" → "12", "000" → "0").

4) Xotirani bo'shatish. Har bir tsiklda ikkala son va yig'indi `free` qilinadi va pointerlar `NULL` ga tenglashtiriladi (osilib qolgan pointerdan saqlanish). Xotira yetmasa dastur xabar chiqarib, allaqachon ajratilganini bo'shatib chiqadi:

```
free(a);
free(b);
free(yig);
a = b = yig = NULL; /* osilib qolgan pointerdan saqlanish */
```

Test natijalari

1-test: 105 xonali son + 1 (ko'chirish butun sonni bo'ylab o'tadi)

```
● ● ● ./main
=== KATTA SONLAR YIG'INDISI ===
1. Ikki sonni qo'shish
2. Chiqish
Tanlang (1-2): 1
1-son: 999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999999
2-son: 1
1-son uzunligi: 105 xona
2-son uzunligi: 1 xona
Yig'indi (106 xona):
100000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
Manzillar: 1-son = 0x55d9b3541af0, 2-son = 0x55d9b35416b0, yig'indi = 0x55d9b3541c00

=== KATTA SONLAR YIG'INDISI ===
1. Ikki sonni qo'shish
2. Chiqish
Tanlang (1-2): 2
Xotira bo'shatildi. Dastur tugadi.
```

2-test: ikkita 111 xonali son

```
● ● ● ./main
=== KATTA SONLAR YIG'INDISI ===
1. Ikki sonni qo'shish
2. Chiqish
Tanlang (1-2): 1
1-son: 123456789012345678901234567890123456789012345678901234567890123456789012345678901234567890123456789012345678901
2-son: 987654321098765432109876543210987654321098765432109876543210987654321098765432109876543210987654321098765432109
1-son uzunligi: 111 xona
2-son uzunligi: 111 xona
Yig'indi (112 xona):
1111111101111111110111111111101111111110111111111011111111101111111110111111111011111111101111111110111111111010
Manzillar: 1-son = 0x55ff318c9af0, 2-son = 0x55ff318c9b80, yig'indi = 0x55ff318c9c90

=== KATTA SONLAR YIG'INDISI ===
1. Ikki sonni qo'shish
2. Chiqish
Tanlang (1-2): 2
Xotira bo'shatildi. Dastur tugadi.
```

3-test: XATO kiritishlar (menyu 7; harf, minus, bo'sh qator; boshidagi nollar)

```
● ● ● ./main
=== KATTA SONLAR YIG'INDISI ===
1. Ikki sonni qo'shish
2. Chiqish
Tanlang (1-2): 7
Xato: 1 yoki 2 kiriting.

=== KATTA SONLAR YIG'INDISI ===
1. Ikki sonni qo'shish
2. Chiqish
Tanlang (1-2): 1
1-son: abc
Xato: faqat raqamlar (0-9) kiriting, bo'sh bo'lmasin, ishora va bo'shliqsiz.
1-son: -5
Xato: faqat raqamlar (0-9) kiriting, bo'sh bo'lmasin, ishora va bo'shliqsiz.
1-son:
Xato: faqat raqamlar (0-9) kiriting, bo'sh bo'lmasin, ishora va bo'shliqsiz.
1-son: 000
2-son: 0012
1-son uzunligi: 3 xona
2-son uzunligi: 4 xona
Yig'indi (2 xona):
12
Manzillar: 1-son = 0x5571f2b8c6b0, 2-son = 0x5571f2b8caf0, yig'indi = 0x5571f2b8cb40

=== KATTA SONLAR YIG'INDISI ===
1. Ikki sonni qo'shish
2. Chiqish
Tanlang (1-2): 2
Xotira bo'shatildi. Dastur tugadi.
```

Test	Kiritish	Kutilgan natija	Natija
1	99...9 (105 ta to'qqiz) + 1	1 va 105 ta nol (106 xona)	To'g'ri

Test	Kiritish	Kutilgan natija	Natija
2	123456789012...901 + 987654321098...109 (111 xona)	Yig'indi 112 xona; Python `int` bilan solishtirilganda bir xil	To'g'ri
3	menyu 7; abc, -5, bo'sh qator; 000 + 0012	Barcha xato kiritishlar rad etiladi; yig'indi 12	To'g'ri

Valgrind natijasi

Xotira oqishi tekshiruvi WSL2/Ubuntu muhitida valgrind bilan bajarildi. Dastur 111 xonali ikki son qo'shilib, keyin menyudan chiqilgan holda ishga tushirildi:

```
gcc -Wall -Wextra -g -o main main.c
valgrind --leak-check=full ./main 2> valgrind.txt
cat valgrind.txt
```

valgrind.txt

```
==492== Memcheck, a memory error detector
==492== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==492== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==492== Command: ./main
==492==
==492==
==492== HEAP SUMMARY:
==492==     in use at exit: 0 bytes in 0 blocks
==492==   total heap usage: 12 allocs, 12 frees, 8,930 bytes allocated
==492==
==492== All heap blocks were freed -- no leaks are possible
==492==
==492== For lists of detected and suppressed errors, rerun with: -s
==492== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Natijada `All heap blocks were freed -- no leaks are possible` qatori bor: 12 ta xotira bloki ajratilgan (2 ta kiritilgan son, yig'indi, vaqtinchalik `tmp`, menyu satrlari, `realloc` bilan kengaytirilgan buferlar va stdio buferi) va 12 ta bo'shatilgan. Xatolar soni: 0.

Har bir malloc qayerda bo'shatiladi: `qator_oqish` ichidagi bufer (kiritilgan son satri) tsikl oxirida `free(a)` va `free(b)` bilan; `katta_qosh` ichidagi `tmp` funksiya ichida, ishi tugagach darhol; `natija` (yig'indi) tsikl oxirida `free(yig)` bilan; menyu uchun o'qilgan satr `menyu_oqish` ichida tekshirilgach. `free` dan keyin pointer hamon eski manzilni saqlab qoladi (osilib qolgan pointer), shuning uchun u NULL ga tenglashtiriladi.

Xulosa (2-topshiriq)

Ishda dinamik xotira bilan ishlash o'rganildi: noma'lum uzunlikdagi satr `malloc/realloc` bilan o'qildi, har bir natija NULL ga tekshirildi, yig'indi yangi ajratilgan satrda qaytarildi va barcha xotira bo'shatildi (valgrind: oqish yo'q). `char **` orqali funksiya argumentni o'zgartirdi. 100 dan ortiq xonali sonlar to'g'ri qo'shildi, noto'g'ri kiritishlar xabar bilan rad etildi.

GitHub repozitoriy: https://github.com/<username>/OS_T2_V18_Xolboyev

Ilova A. Funksiyalar izohi (himoya uchun)

1-topshiriq (siklli kalkulyator)

Funksiya	Nima qiladi	Nega shunday
`qosh, ayir, kopaytir, bol`	Mos arifmetik amalni bajaradi	Hammasi bir xil imzoda, shuning uchun bitta funksiya pointer turiga mos
`qoldiq(a, b)`	fmod(a, b) ni qaytaradi	C da % faqat butun sonlar bilan ishlaydi; double uchun fmod kerak (-lm)
`amal_tanla(belgi)`	Belgiga qarab funksiya pointerini qaytaradi, noto'g'ri belgida NULL	switch bilan tanlash; NULL qaytishi belgini tekshirish uchun ham ishlatiladi
`qator_oqish(...)`	fgets bilan qator o'qiydi, '\n' ni olib tashlaydi, uzun qatorni rad etadi	scanf bufer chegarasini saqlamaydi; fgets saqlaydi
`son_oqish(...)`	strtod bilan son o'qiydi; q kiritilsa 2 qaytaradi; harf, nan/inf rad etiladi	Xato kiritishda dastur qotib qolmaydi
`amal_oqish(...)`	Amal belgisini o'qib tekshiradi	Faqat + - * / % qabul qilinadi
`main()`	Siklda sonlar va amalni oladi, nolga bo'lishni tekshiradi, pointer orqali chaqiradi, %p manzillarini chiqaradi	q kiritilganda sikl tugaydi

2-topshiriq (katta sonlar)

Funksiya	Nima qiladi	Nega shunday
`qator_oqish(char **satr)`	Noma'lum uzunlikdagi qatorni malloc/realloc bilan o'qiydi, manzilni *satr ga yozadi	Uzunlik oldindan noma'lum; char ** orqali chaqiruvchining pointeri o'zgaradi
`raqamlarmi(s)`	Satr bo'sh emas va faqat raqamlardan iboratligini tekshiradi	Harf, minus, bo'shliq rad etiladi
`nolsiz(s)`	Boshidagi ortiqcha nollardan o'tib ketadi	'0012' va '12' bir xil son
`katta_qosh(a, b)`	Ustun usulida qo'shadi, natijani YANGI satrda qaytaradi	Son int/long long ga sig'maydi; chaqiruvchi free qilishi shart
`son_oqish(...)`	To'g'ri son kelguncha qayta so'raydi	Xato kiritishdan keyin xotira oqmasligi uchun noto'g'ri satr free qilinadi
`menyu_oqish()`	1 yoki 2 ni o'qiydi	Menyu qatori ham dinamik, har gal free qilinadi
`main()`	Siklda ikki son o'qiydi, qo'shadi, natijani chiqaradi, hammasini free qiladi	Har bir malloc ning free si bor

Ilova B. Himoya savollariga javoblar

Nima uchun % uchun fmod() kerak?

C da % operatori faqat butun sonlar bilan ishlaydi. double uchun math.h dagi fmod() ishlatiladi va kompilyatsiyada -lm qo'shiladi.

Funksiya pointeri qanday e'lon qilinadi?

double (*amal)(double, double); bu 'ikkita double olib double qaytaradigan funksiyaga pointer'. Qavslar muhim: ularsiz bu double * qaytaradigan funksiya e'loni bo'ladi.

amal(a, b) va (*amal)(a, b) farqi bormi?

Yo'q, ikkalasi ham bir xil. Funksiya pointeri chaqirilganda * yozish shart emas.

fmod(-7, 3) nima uchun -1?

fmod natijasi birinchi sonning ishorasini saqlaydi (C dagi % kabi). -7 = -2*3 + (-1).

Nolga bo'lish nima uchun funksiyadan oldin tekshiriladi?

Butun sonlarda nolga bo'lish dastur qulashiga olib keladi; double da inf/nan chiqadi. Oldindan tekshirib, foydalanuvchiga tushunarli xabar beriladi.

scanf da nima uchun &kerak;? Nima uchun fgets ishlatildi?

scanf o'zgaruvchining MANZILINI kerak qiladi, chunki u qiymatni shu manzilga yozadi. fgets bufer hajmini hisobga oladi va xato kiritishda dastur qotib qolmaydi.

.o fayl va bajariluvchi fayl farqi?

Linking bosqichida linker kutubxonalarni bog'laydi, tashqi funksiyalar manzillarini to'ldiradi va ishga tushirish kodini (_start) qo'shadi. .o fayl esa bajarib bo'lmaydi.

Kodingizdagi har bir malloc qayerda bo'shatiladi? free dan keyin pointer nima bo'ladi?

Yuqoridagi 'Valgrind natijasi' bo'limida yozilgan. free dan keyin pointer eski manzilni saqlaydi (osilib qolgan pointer), shuning uchun NULL ga tenglashtiriladi.

Nima uchun realloc natijasi alohida o'zgaruvchiga olinadi?

realloc NULL qaytarsa, eski bufer hali ham mavjud va uni free qilish kerak. Natijani to'g'ridan-to'g'ri buf ga yozsak, eski manzil yo'qolib, xotira oqadi.

char ** nima uchun kerak?

Funksiya yangi ajratilgan satr manzilini chaqiruvchining pointeriga yozishi kerak. C da argumentlar nusxa bo'lib uzatiladi, shuning uchun pointerning o'zini o'zgartirish uchun uning manzili (char **) beriladi.

Katta son nima uchun satr sifatida saqlanadi?

100+ xonali son hech bir butun son turiga (long long ~19 xona) sig'maydi, shuning uchun har bir raqam satrda alohida belgi bo'lib turadi va qo'lda qo'shiladi.

Ilova C. 1-topshiriq: main.c (to'liq kod)

Repozitoriy: https://github.com/<username>/OS_T1_V18_Xolboyev

```
/*
 * Operatsion tizimlar fani | 1-topshiriq | Variant 18
 * Mavzu : Siklli kalkulyator (+ - * / %)
 *
 * 1-masala (C asoslari) : siklli kalkulyator, kiritishni tekshirish, nolga bo'lish tekshiruvi.
 * 2-masala (pointerlar) : amal belgisiga qarab funksiya pointeri tanlanadi:
 *                         double (*amal)(double, double)
 *
 * Kompilyatsiya: gcc -Wall -Wextra -o main main.c -lm
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <errno.h>

#define BUF 128 /* kiritish buferi hajmi (statik massiv) */

/* ---------- Amallar (funksiya pointeri shu imzoga ega) ---------- */
static double qosh(double a, double b)     { return a + b; }
static double ayir(double a, double b)     { return a - b; }
static double kopaytir(double a, double b) { return a * b; }
static double bol(double a, double b)      { return a / b; }
/* C da % double bilan ishlamaydi, shuning uchun fmod() */
static double qoldiq(double a, double b)   { return fmod(a, b); }

/* Amal belgisiga mos funksiya pointerini qaytaradi; noto'g'ri belgi bo'lsa NULL. */
static double (*amal_tanla(char belgi))(double, double)
{
    switch (belgi) {
    case '+': return qosh;
    case '-': return ayir;
    case '*': return kopaytir;
    case '/': return bol;
    case '%': return qoldiq;
    default:  return NULL;
    }
}

/* ---------- Kiritish ---------- */
```

```
/* Qator o'qiydi va '\n' ni olib tashlaydi. 1 - ok, 0 - juda uzun, -1 - EOF. */
static int qator_oqish(const char *taklif, char *buf, int hajm)
{
    size_t n;
    printf("%s", taklif);
    if (fgets(buf, hajm, stdin) == NULL)
        return -1;
    n = strlen(buf);
    if (n > 0 && buf[n - 1] == '\n') {
        buf[n - 1] = '\0';
    } else {
        int c;
        while ((c = getchar()) != '\n' && c != EOF)
            ;
        printf("Xato: juda uzun kiritish.\n");
        return 0;
    }
    n = strlen(buf);
    if (n > 0 && buf[n - 1] == '\r')
        buf[n - 1] = '\0';
    return 1;
}

/*
 * Son o'qiydi. Qaytish: 1 - son, 2 - foydalanuvchi 'q' kiritdi, -1 - EOF.
 * Harf, bo'sh qator, nan/inf va oraliqdan tashqari qiymat rad etiladi.
 */
static int son_oqish(const char *taklif, double *natija)
{
    char buf[BUF];
    char *oxiri;
    double v;
    int r;

    for (;;) {
        r = qator_oqish(taklif, buf, BUF);
        if (r == -1)
            return -1;
        if (r == 0)
            continue;
        if (strcmp(buf, "q") == 0 || strcmp(buf, "Q") == 0)
            return 2;
        errno = 0;
        v = strtod(buf, &oxiri);
        while (*oxiri == ' ' || *oxiri == '\t')
            oxiri++;
        if (oxiri == buf || *oxiri != '\0' || !isfinite(v)) {
            printf("Xato: son kiriting (masalan 12, -3.5) yoki q.\n");
            continue;
        }
        if (errno == ERANGE) {
            printf("Xato: son juda katta.\n");
            continue;
        }
        *natija = v;
        return 1;
    }
}

/* Amal belgisini o'qiydi. 1 - ok, -1 - EOF. */
static int amal_oqish(char *belgi)
{
    char buf[BUF];
    int r;

    for (;;) {
        r = qator_oqish("Amal (+ - * / %): ", buf, BUF);
        if (r == -1)
            return -1;
        if (r == 0)
            continue;
        if (strlen(buf) != 1 || amal_tanla(buf[0]) == NULL) {
            printf("Xato: amal faqat + - * / %% dan biri bo'lishi kerak.\n");
            continue;
        }
        *belgi = buf[0];
        return 1;
    }
}
```

```
/* ---------- Asosiy dastur ---------- */
int main(void)
{
    double a, b, natija;
    char belgi;
    int r;
    double (*amal)(double, double); /* funksiya pointeri */

    printf("=== SIKLLI KALKULYATOR ===\n(to'xtatish uchun son o'rniga q kiriting)\n");
    for (;;) {
        r = son_oqish("\nBirinchi son: ", &a);
        if (r != 1)
            break;
        r = son_oqish("Ikkinchi son: ", &b);
        if (r != 1)
            break;
        if (amal_oqish(&belgi) != 1)
            break;

        /* nolga bo'lish tekshiruvi (/ va % uchun) */
        if ((belgi == '/' || belgi == '%') && b == 0.0) {
            printf("Xato: nolga bo'lish mumkin emas.\n");
            continue;
        }

        amal = amal_tanla(belgi);   /* pointer amal belgisiga qarab tanlanadi */
        natija = amal(a, b);        /* funksiya pointer orqali chaqiriladi */

        printf("Natija: %g %c %g = %g\n", a, belgi, b, natija);
        printf("--- Xotira manzillari ---\n");
        printf("natija o'zgaruvchisi &natija = %p\n", (void *)&natija);
        printf("pointer o'zgaruvchisi &amal   = %p\n", (void *)&amal);
    }
    printf("Dastur tugadi.\n");
    return 0;
}
```

A qism: hello.c

```
#include <stdio.h>
int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

Ilova D. 2-topshiriq: main.c (to'liq kod)

Repozitoriy: https://github.com/<username>/OS_T2_V18_Xolboyev

```
/*
 * Operatsion tizimlar fani | 2-topshiriq | Variant 18
 * Mavzu : Katta sonlar (100+ xonali) yig'indisi - satr sifatida, dinamik xotirada
 *
 * Ikkita katta son satr sifatida o'qiladi (uzunligi oldindan noma'lum, realloc bilan
 * o'sadi), yig'indisi yangi dinamik satrda qaytariladi. Oxirida barcha xotira free qilinadi.
 *
 * Kompilyatsiya: gcc -Wall -Wextra -g -o main main.c
 * Tekshiruv    : valgrind --leak-check=full ./main
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BOSH_HAJM 32 /* o'qish buferining boshlang'ich hajmi (keyin 2 barobar o'sadi) */

/*
 * Standart oqimdan butun qatorni o'qiydi. Xotira dinamik ajratiladi, kerak bo'lsa
 * realloc bilan kengayadi. Natija *satr ga yoziladi (ARGUMENT POINTER ORQALI O'ZGARADI).
 * Qaytish: 1 - ok, 0 - xotira yetmadi, -1 - EOF (hech narsa o'qilmadi).
 */
static int qator_oqish(char **satr)
{
    size_t sigim = BOSH_HAJM, n = 0;
    char *buf = malloc(sigim);
    int c;

    if (buf == NULL)
        return 0;
    while ((c = getchar()) != '\n' && c != EOF) {
        if (n + 1 >= sigim) {
            char *yangi = realloc(buf, sigim * 2);
            if (yangi == NULL) {
                free(buf);
                return 0;
            }
            buf = yangi;
            sigim *= 2;
        }
        buf[n++] = (char)c;
```

```
    }
    if (c == EOF && n == 0) {
        free(buf);
        return -1;
    }
    if (n > 0 && buf[n - 1] == '\r')
        n--;
    buf[n] = '\0';
    *satr = buf;
    return 1;
}

/* Satr faqat raqamlardan iborat va bo'sh emasmi? */
static int raqamlarmi(const char *s)
{
    const char *p = s;
    if (*p == '\0')
        return 0;
    for (; *p != '\0'; p++)
        if (*p < '0' || *p > '9')
            return 0;
    return 1;
}

/* Boshidagi ortiqcha nollarni o'tkazib yuboradi ("000123" -> "123"; "000" -> "0"). */
static const char *nolsiz(const char *s)
{
    while (*s == '0' && *(s + 1) != '\0')
        s++;
    return s;
}

/*
 * Ikki katta son yig'indisi. Natija YANGI ajratilgan satrda qaytariladi
 * (chaqiruvchi free qilishi shart). Xotira yetmasa NULL.
 */
static char *katta_qosh(const char *a, const char *b)
{
    size_t la, lb, uzun, i;
    int carry = 0;
```

```
    char *tmp, *natija;
    const char *bosh;

    a = nolsiz(a);
    b = nolsiz(b);
    la = strlen(a);
    lb = strlen(b);
    uzun = (la > lb ? la : lb) + 1; /* +1: oxirgi ko'chirish uchun */

    tmp = malloc(uzun + 1); /* teskari tartibda yig'iladi */
    if (tmp == NULL)
        return NULL;

    for (i = 0; i < uzun; i++) {
        int s = carry;
        if (i < la)
            s += *(a + la - 1 - i) - '0';
        if (i < lb)
            s += *(b + lb - 1 - i) - '0';
        *(tmp + i) = (char)('0' + s % 10);
        carry = s / 10;
    }
    *(tmp + uzun) = '\0';

    /* teskari tartibni to'g'rilash va boshidagi nolni tashlash */
    natija = malloc(uzun + 1);
    if (natija == NULL) {
        free(tmp);
        return NULL;
    }
    for (i = 0; i < uzun; i++)
        *(natija + i) = *(tmp + uzun - 1 - i);
    *(natija + uzun) = '\0';
    free(tmp);

    bosh = nolsiz(natija);
    if (bosh != natija)
        memmove(natija, bosh, strlen(bosh) + 1);
    return natija;
}
/* Raqamlardan iborat son o'qiydi; noto'g'ri bo'lsa qayta so'raydi. 1 - ok, 0/-1 - xato/EOF. */
static int son_oqish(const char *taklif, char **son)
{
    for (;;) {
        int r;
        printf("%s", taklif);
        r = qator_oqish(son);
        if (r != 1)
            return r;
        if (raqamlarmi(*son))
            return 1;
        printf("Xato: faqat raqamlar (0-9) kiriting, bo'sh bo'lmasin, ishora va bo'shliqsiz.\n");
        free(*son);
        *son = NULL;
    }
}

/* Menyu tanlovi: 1 yoki 2. */
static int menyu_oqish(void)
{
    for (;;) {
        char *s = NULL;
        int r, v = 0;
        printf("\n=== KATTA SONLAR YIG'INDISI ===\n1. Ikki sonni qo'shish\n2. Chiqish\nTanlang (1-2): ");
        r = qator_oqish(&s);
        if (r != 1)
            return -1;
        if (strcmp(s, "1") == 0) v = 1;
        else if (strcmp(s, "2") == 0) v = 2;
        free(s);
        if (v != 0)
            return v;
        printf("Xato: 1 yoki 2 kiriting.\n");
    }
}

int main(void)
{
    int tanlov;
```

```
    while ((tanlov = menyu_oqish()) == 1) {
        char *a = NULL, *b = NULL, *yig = NULL;
        int r;

        r = son_oqish("1-son: ", &a);
        if (r == 1)
            r = son_oqish("2-son: ", &b);
        if (r == 0) {
            printf("Xato: xotira yetarli emas!\n");
            free(a);
            return 1;
        }
        if (r == -1) {
            free(a);
            free(b);
            break;
        }

        yig = katta_qosh(a, b);
        if (yig == NULL) {
            printf("Xato: xotira yetarli emas!\n");
            free(a);
            free(b);
            return 1;
        }
        printf("1-son uzunligi: %zu xona\n2-son uzunligi: %zu xona\n", strlen(a), strlen(b));
        printf("Yig'indi (%zu xona):\n%s\n", strlen(yig), yig);
        printf("Manzillar: 1-son = %p, 2-son = %p, yig'indi = %p\n",
               (void *)a, (void *)b, (void *)yig);

        free(a);
        free(b);
        free(yig);
        a = b = yig = NULL; /* osilib qolgan pointerdan saqlanish */
    }
    printf("Xotira bo'shatildi. Dastur tugadi.\n");
    return 0;
}
```